

```
Script started on 2026-02-15 01:43:27-06:00 [TERM="xterm" TTY="/dev/pts/1" COLUMNS=
rc47379@ares:~$ pwd

/home/students/rc47379
rc47379@ares:~$ cat stringtranslation.info

NAME: Calvin Rice

CLASS: CSC122-002

TITLE: String Translation

LEVEL: 2

DESCRIPTION: Turns a string of numbers into a long

rc47379@ares:~$ show-code stringtranslation.cpp

stringtranslation.cpp:

1  #include <iostream>
2  #include "strextra.h"
3  #include <string>
4
5  int main()
6  {
7      std::cout << "1234 = " << str_to_long("1234") << std::endl;
8      std::cout << "-123 = " << str_to_long("-123") << std::endl;
9      std::cout << "1 = " << str_to_long("1") << std::endl;
10     std::cout << "-1 = " << str_to_long("-1") << std::endl;
11     std::cout << "+1 = " << str_to_long("+1") << std::endl;
12     std::cout << "+1.4 = " << str_to_long("+1.4") << std::endl;
13     std::cout << "apes = " << str_to_long("apes") << std::endl;
14     std::cout << "2 boxes = " << str_to_long("2 boxes") << std::endl;
15     std::cout << "12false = " << str_to_long("12false") << std::endl;
16     std::cout << "-12.3 = " << str_to_long("-12.3") << std::endl;
17 }

rc47379@ares:~$ show-code strextra.h

strextra.h:

1  #ifndef STRING_TRANSLATOR
2  #define STRING_TRANSLATOR
3  #include <string>
4
5  long str_to_long(std::string input);
6
7  #endif
```

```
rc47379@ares:~$ show-code strextra.cpp

strextra.cpp:

1  #include "strextra.h"
2  #include <string>
3  #include <iostream>
4  #include <ctype.h>
5
6  using namespace std;
7
8  long str_to_long(std::string input)
9  {
10     long output = 0;
11     bool positive = input[0] != '-';
12
13     for(int i = 0; i <= input.length(); i++)
14     {
15         if(input[i] == '.')
16         {
17             break;
18         }
19
20         if(isdigit(input[i]))
21         {
22             output = output * 10 + (input[i] - '0');
23         }
24     }
25
26     if(!positive)
27     {
28         output *= -1;
29     }
30
31     return output;
32 }

rc47379@ares:~$ CPP stringtranslation strextra

strextra.cpp...
stringtranslation.cpp***
strextra.cpp: In function 'long int
str_to_long(std::string)':
strextra.cpp:13:26: warning: comparison
of integer expressions of different signedness: 'int'
and 'std::__cxx11::basic_string<char>::size_type'
{aka 'long unsigned int'}
[-Wsign-compare]
13 |         for(int i = 0; i <= input.length();
    |         i++)
    |
```

```
rc47379@ares:~$ ./stringtranslation.out
```

```
1234 = 1234
-123 = -123
1 = 1
-1 = -1
+1 = 1
+1.4 = 1
apes = 0
2 boxes = 2
12false = 12
-12.3 = -12
```

```
rc47379@ares:~$ cat stringtranslation.tpq
```

What argument(s) does your function need? Will you change the argument(s)? What type?

-The function only needs the string input, everything else is inferred from the string

How can you easily skip past the beginning whitespace? Is there a library function?

-I can treat the beginning whitespace as just another non-digit character and skip it

Is the translation of "-123" actually significantly different than that for "+123" or not?

-The only thing that changes is the - sign is recognized and used to make the output negative

When translating "1" how can you access just that character '1'? What other characters are there?

-The end of string character is present but it is skipped since it isn't a digit

When translating "1" how can you convert the character representation of the digit to an integer?

-Subtracting '0', which has an ascii value of 48, makes the characters line up with the digits

When translating "12", how can you ensure that you end up with 12 instead of 3? (i.e. how do you handle multiple digits?)

-I multiply the resulting long by 10 each time I add another digit to make sure the digits are in the right place

How can you stop and ignore the extra information in "+1.4"? (i.e. the ".4" is irrelevant)

-The + and letters are skipped since they aren't digits with the isdigit() function

How can you tell the caller the position at which you stopped translating? (Hint: use a pointer)

-The loop already tracks which place in the string the program is, so I could return that

When the string contains no integer, how can you have the user's value be/remain unchanged?

-The returned long is just the default constructed long, which is 0

How can the user detect that their value hasn't been modified? (i.e. when their string doesn't contain a number)

-The program would have looped through every character if there were no numbers in the string

When translating a string to an integer, we can start from either the front or the back

-I initially started from the end of the string since I could have multiplied each digit by the correct power of 10

How do you protect your library from being circularly included?

=#ifndef, #define and #endif with a unique name

What changes are needed in your main application (the test application here) to get it to use the library?

-The main application needs to include the header file of the library

How many files does your library consist of? What are they? Which one(s) do you #include?

-The library has the header and definition files and the main application and the test application

What does the exit function do? How do you use it?

Script done on 2026-02-15 01:44:32-06:00 [COMMAND_EXIT_CODE="0"]